

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS
GERAIS

Engenharia de Computação

ARQUITETURA DE COMPUTADORES
III

Pipeline Escalar e Superescalar

Pablo Rodrigues de oliveira
Paulo Henrique Rodrigues Neves
Thiago Marques Lopes Coelho

Belo Horizonte
2026

Pablo Rodrigues de oliveira
Paulo Henrique Rodrigues Neves
Thiago Marques Lopes Coelho

TRABALHO PRÁTICO

Trabalho prático apresentado à disciplina de Arquitetura de Computadores III do curso de Engenharia de Computação da Pontifícia Universidade Católica de Minas Gerais, como requisito parcial para obtenção de nota.

Professor: Ricardo Carlini Sperandio

Belo Horizonte
2026

Sumário

1	Resumo	3
2	Introdução	4
3	Fundamentação	4
3.1	Pipeline Clássico de 5 Estágios	4
3.2	Conflitos de Pipeline (Hazards)	4
3.2.1	Hazards de Dados	5
3.2.2	Hazards de Controle	5
3.3	Mecanismos de Mitigação: Forwarding e Predição	5
3.3.1	Forwarding (Adiantamento)	5
3.3.2	Estratégias de Predição de Desvios	5
3.4	Arquiteturas Superescalares e Execução Dinâmica	5
3.4.1	Algoritmo de Tomasulo	6
3.4.2	Renomeação de Registradores	6
3.4.3	Reorder Buffer (ROB)	6
4	Metodologia	6
5	Resultados A	7
5.1	Análise da Sequência S2: Loop e Variável de Indução	15
5.1.1	Identificação de Hazards e Dependências	15
5.1.2	Otimização via Escalonamento Estático de Código	16
5.1.3	Impacto no Desempenho e Hazard de Controle	16
5.2	Análise da Sequência S3: Load-Use Hazard	17
5.2.1	Identificação do Conflito (O Paradoxo Temporal)	17
5.2.2	Demonstração Analítica e Diagrama de Estágios	17
5.2.3	Prova da Insuficiência do Forwarding	17
5.2.4	Solução Proposta: Escalonamento Estático	18
5.3	Análise do Preditor Dinâmico de 2 Bits (Saturating Counter)	18
5.4	Diagrama de Estágios Manual - Sequência S2 (Loop)	19
6	Resultados B	20
6.1	B1. Algoritmo de Tomasulo — Simulação Manual	20
6.2	B.2 Status dos Registradores e Transmissões no CDB	21
6.3	B.3 Questões do Relatório — Análise T2	22
6.4	B.4 Estado das Estações de Reserva — Análise T2	22
6.5	B.5 Status dos Registradores e Transmissões — Análise T2	25
6.6	Questões do Relatório	25

6.7	B.2 Renomeação de Registradores	26
6.8	1. Grafo de Dependência	26
7	Análise Integradora – Do Escalar e Superescalar	33
7.1	B.3 Simulação Superescalar	33
8	Conclusão	36
A	Sequências S1–S5 e Dados Brutos	38
B	Transcrição Completa das Análises — S1 a S5	38
B.1	Sequência S1	38
B.2	Sequência S2	39
B.3	Sequência S3	41
B.4	Sequência S4	42
B.5	Sequência S5	43
B.6	Conclusão Geral das Análises	45

1 Resumo

Este trabalho apresenta uma análise detalhada dos impactos gerados por conflitos de dados e de controle (hazards) no desempenho de processadores com pipelines de cinco estágios, utilizando as sequências de instruções S2 e S3 baseadas na arquitetura MIPS. Investigou-se o comportamento do conflito Load-Use em estruturas aritméticas sequenciais e em loops de processamento de vetores. Demonstrou-se analiticamente a insuficiência física do mecanismo de forwarding isolado para sanar a colisão temporal desse tipo de hazard, o que exige a introdução compulsória de paradas (stalls) pelo hardware. Como contramedida, aplicou-se a técnica de escalonamento estático de instruções via compilador, reordenando o código de forma a preencher os slots de atraso com instruções independentes, eliminando as bolhas e otimizando o CPI. Adicionalmente, avaliou-se a eficácia do preditor dinâmico de desvios de 2 bits baseado em contadores saturados frente à estratégia estática Always Not-Taken, evidenciando como a inércia do histórico reduz drasticamente os descartes de instruções (flushes) em estruturas repetitivas, validando abordagens integradoras para a eficiência do fluxo de execução.

2 Introdução

O desenvolvimento de arquiteturas de computadores modernas apoia-se fortemente na eficiência da segmentação de instruções, conceito conhecido como *pipelining*. Embora essa técnica potencialize a taxa de vazão (*throughput*) do sistema compartilhando recursos funcionais, a execução simultânea de partes distintas de comandos cria cenários de conflitos temporais e lógicos chamados *hazards*. Este relatório tem como objetivo explorar de maneira experimental e analítica o comportamento dessas restrições físicas dentro de uma estrutura clássica de cinco estágios. Por meio do mapeamento e reordenação de fluxos aritméticos e de controle, busca-se compreender os limites de soluções comuns de hardware e software, documentando os ganhos em eficiência obtidos por técnicas de mitigação controlada.

3 Fundamentação

A evolução das arquiteturas de computadores baseia-se em técnicas de paralelismo para otimizar o desempenho do processador. Esta seção aborda os conceitos teóricos fundamentais, partindo dos pipelines escalares até os sistemas superescalares com execução fora de ordem.

3.1 Pipeline Clássico de 5 Estágios

O *pipelining* consiste na sobreposição da execução de múltiplas instruções. O pipeline clássico RISC divide esse processo em cinco estágios isolados por registradores de acoplamento:

- **IF (Instruction Fetch):** Busca da instrução na memória baseada no Contador de Programa (*PC*) e incremento do *PC*.
- **ID (Instruction Decode):** Decodificação da instrução e leitura dos operandos no banco de registradores.
- **EX (Execute):** Execução de operações aritméticas/lógicas pela ULA ou cálculo de endereços de memória.
- **MEM (Memory Access):** Leitura (*Load*) ou escrita (*Store*) de dados na memória.
- **WB (Write Back):** Escrita do resultado final de volta no banco de registradores arquitetais.

3.2 Conflitos de Pipeline (Hazards)

Os *hazards* são situações que interrompem o fluxo ideal do pipeline, forçando paradas (*stalls*). Dividem-se em duas categorias principais neste estudo:

3.2.1 Hazards de Dados

Surgem quando a ordem de acesso aos operandos é violada em relação à execução estritamente sequencial:

- **RAW (Read After Write):** Dependência verdadeira. Uma instrução tenta ler um dado antes que a anterior conclua sua escrita.
- **WAR (Write After Read):** Anti-dependência. Uma instrução posterior tenta escrever em um destino antes que a anterior o leia.
- **WAW (Write After Write):** Dependência de saída. Instruções posteriores tentam escrever no mesmo destino de uma anterior, ameaçando a ordem lógica final.

3.2.2 Hazards de Controle

Ocorrem quando o fluxo do programa é alterado por desvios (*branches*), impedindo a busca correta da próxima instrução até que a condição seja avaliada. Instruções buscadas erroneamente precisam ser descartadas (*flush*).

3.3 Mecanismos de Mitigação: Forwarding e Predição

3.3.1 Forwarding (Adiantamento)

Técnica de hardware que minimiza *stalls* por RAW ao interconectar as saídas intermediárias do pipeline (EX/MEM e MEM/WB) diretamente às entradas da ULA. Permite reutilizar o dado assim que calculado, sem aguardar o estágio WB.

3.3.2 Estratégias de Predição de Desvios

Minimizam os hazards de controle ao antecipar o comportamento dos desvios:

- **Estática (Always Not-Taken):** Assume fixamente que o desvio nunca será tomado, mantendo o fluxo sequencial.
- **Dinâmica de 1 Bit:** Registra o último comportamento do desvio em uma tabela. Tende a oscilar e falhar duas vezes em saídas de loops.
- **Dinâmica de 2 Bits:** Baseia-se em contadores saturados de quatro estados. Exige dois erros consecutivos para alterar a predição, conferindo inércia e maior estabilidade em loops.

3.4 Arquiteturas Superescalares e Execução Dinâmica

Processadores **Superescalares** despacham múltiplas instruções por ciclo duplicando suas unidades funcionais.

3.4.1 Algoritmo de Tomasulo

Habilita a ****Execução Fora de Ordem Dinâmica**** (*Out-of-Order*). As instruções decodificadas aguardam em buffers chamados **Estações de Reserva** (*Reservation Stations*) até que seus operandos fiquem prontos. Ao término da execução, os resultados são transmitidos para todo o sistema via Barramento Comum de Dados (*CDB*), mitigando stalls por RAW.

3.4.2 Renomeação de Registradores

Elimina dependências falsas (WAR e WAW) ao mapear dinamicamente os registradores arquiteturais lógicos em tags das Estações de Reserva ou registradores físicos através de uma **RAT** (*Register Alias Table*).

3.4.3 Reorder Buffer (ROB)

Permite a execução fora de ordem, mas garante o **Commit** (retirada) estritamente em ordem (*in-order*). O ROB armazena os resultados especulativos em uma fila circular sequencial, preservando o estado preciso do sistema diante de exceções e desvios incorretos.

4 Metodologia

A avaliação de desempenho do sistema realizou-se por meio de plataformas de simulação computacional dedicadas a pipelines MIPS. Foram testadas cinco configurações arquiteturais modulares com perfis escalares e superescalares incrementais:

- **C1 (Baseline):** Escalar padrão de duas vias (*2-wide*), com despacho e retirada puramente em ordem, sem adiantamento avançado.
- **C2:** Modelo *2-wide* operando com despacho em ordem e retirada fora de ordem, contendo suporte a execução especulativa simples.
- **C3:** Modelo superescalar *2-wide* com execução fora de ordem (*Out-of-Order*) dinâmica assistida por um Reorder Buffer de 16 entradas (ROB-16).
- **C4:** Modelo superescalar largo de quatro vias (*4-wide*) com execução dinâmica OoO assistida por uma janela de ROB-32 entradas.
- **C5:** Configuração avançada de quatro vias *4-wide* OoO com ROB-32 acoplada a um Preditor Dinâmico de Desvios de 2 bits.

As cargas de trabalho aplicadas compreendem as sequências customizadas de teste de S1 a S5.

5 Resultados A

A.3 — Roteiro Experimental

1. Identificação dos Hazards

A sequência apresenta exclusivamente hazards de dados do tipo **RAW** (*Read After Write*).

Tabela 1: Mapeamento de Dependências RAW no Código S1

Instrução Consumidora	Registrador Dependente	Produzido por
add \$t1, \$t0, \$t0	\$t0	addiu
add \$t2, \$t1, \$t0	\$t1	add \$t1
add \$t3, \$t2, \$t1	\$t2	add \$t2
add \$t4, \$t3, \$t2	\$t3	add \$t3
add \$t5, \$t4, \$t3	\$t4	add \$t4
sub \$t6, \$t5, \$t4	\$t5	add \$t5

Todas as dependências possuem distância 1, ou seja, a instrução consumidora utiliza imediatamente o resultado da instrução anterior.

Configuração (a) Sem Forwarding e Sem Predição

Comportamento: Sem *forwarding*, o valor produzido só pode ser utilizado após o estágio WB (*Write Back*). Como cada instrução depende diretamente da anterior, o pipeline precisa inserir bolhas (*bubbles*), onde cada RAW gera *stalls*.

Tabela 2: Análise de Stalls na Configuração Sem Forwarding

Pipeline: Instrução	Stall Necessário
add \$t1 depende de addiu	2 ciclos
add \$t2 depende de add \$t1	2 ciclos
add \$t3 depende de add \$t2	2 ciclos
add \$t4 depende de add \$t3	2 ciclos
add \$t5 depende de add \$t4	2 ciclos
sub \$t6 depende de add \$t5	2 ciclos
Total Aproximado	12 stalls RAW

CPI Estimado:

- Número total de instruções: 9 instruções.

- Ciclos aproximados:
 - Pipeline ideal: $N + 4 = 13$ ciclos.
 - Stalls adicionais: 12 ciclos.
 - Total: 25 ciclos.
- $CPI = \frac{25}{9} \approx 2,78$ CPI.

Configuração (b) Com Forwarding Completo

Comportamento: Com *forwarding*, o resultado da ALU pode ser encaminhado diretamente do estágio EX/MEM, eliminando-se praticamente todos os *stalls* RAW entre operações aritméticas. Como não há instruções *lw*, o adiantamento resolve completamente os hazards.

Tabela 3: Resolução de Hazards com Forwarding

Hazard	Resolvido por Forwarding?
RAW em \$t0	Sim
RAW em \$t1	Sim
RAW em \$t2	Sim
RAW em \$t3	Sim
RAW em \$t4	Sim
RAW em \$t5	Sim

Onde nenhum *Stall* seria necessário.

CPI Estimado: 9 instruções e 13 ciclos totais $\rightarrow CPI = \frac{13}{9} \approx 1,44$ CPI.

Configuração (c) Forwarding + Predição Estática

Não existem *branches* na sequência S1. Logo, a predição não produz efeito e o desempenho permanece igual ao caso (b).

- $CPI \approx 1,44$ CPI.

Configuração (d) Forwarding + Predição Dinâmica de 1 Bit

A sequência não possui desvios condicionais. Portanto, nenhum benefício adicional é gerado e o CPI mantém-se idêntico aos casos anteriores (c) e (b).

- $CPI \approx 1,44$ CPI.

Configuração (e) Forwarding + Predição Dinâmica de 2 Bits

Também não há impacto pelo mesmo motivo da ausência de desvios condicionais.

Tabela 4: Tabela Consolidada do Programa S1

Configuração	Forwarding	Predição	Stalls	CPI Aproximado
(a)	Não	Não	12	2,78
(b)	Sim	Não	0	1,44
(c)	Sim	Estática	0	1,44
(d)	Sim	1 bit	0	1,44
(e)	Sim	2 bits	0	1,44

A.4 — Análise Requerida

1. **Hazard Dominante:** RAW (*Read After Write*). A sequência foi construída especificamente para produzir dependências RAW consecutivas de distância 1.
2. **Técnica Mais Eficaz:** A técnica mais eficaz foi o *forwarding completo*. O adiantamento elimina integralmente os *stalls* porque todas as instruções são operações aritméticas, os resultados ficam disponíveis ao final do estágio EX e o dado pode ser encaminhado diretamente à próxima instrução.
3. **Speedup:** Comparando (a) e (b):

$$\text{Speedup} = \frac{\text{CPI}_{(a)}}{\text{CPI}_{(b)}} = \frac{2,78}{1,44} \approx 1,93$$

Ou seja, o *forwarding* quase dobra o desempenho da sequência.

4. **Análise da Fórmula de Desempenho:**

$$\text{Tempo} = N_{\text{inst}} \times \text{CPI} \times T_{\text{ciclo}}$$

Nesta sequência, N_{inst} e T_{ciclo} permanecem constantes; somente o CPI é afetado. O *forwarding* reduz drasticamente o CPI ao eliminar *stalls*.

5. **Discussão Técnica:** A sequência S1 demonstra claramente a importância do *forwarding* em pipelines escalares. Sem ele, cada dependência RAW força a interrupção temporária do pipeline, derrubando o *throughput*. Com o *forwarding*, os resultados são reutilizados imediatamente e o pipeline aproxima-se do desempenho ideal. Como a sequência não possui *loads*, *stores* ou *branches*, não há influência de hazards estruturais, de controle

ou de *load-use*. Assim, S1 representa um caso clássico onde o *forwarding* praticamente elimina toda penalidade.

A.3 — Roteiro Experimental (S2)

1. Identificação dos Hazards

- **Hazard 1 — Load-use RAW:**

```
lw    $t3, 0($t2)
add   $t0, $t0, $t3
```

O `add` usa imediatamente o valor carregado pelo `lw`.

- **Hazard 2 — RAW na variável de indução:**

```
addiu $t1, $t1, -1
bne   $t1, $zero, loop
```

O `bne` (branch) depende imediatamente do novo valor calculado em `$t1`.

- **Hazard 3 — Controle:**

```
bne $t1, $zero, loop
```

O processador não sabe imediatamente se o branch será tomado e qual será o próximo PC. Logo, podem ocorrer *flushes* ou *stalls* de branch.

Configuração (a) Sem Forwarding e Sem Predição

Comportamento: Sem *forwarding*, o `add` precisa esperar o `lw` concluir o estágio WB, e o `bne` precisa esperar a atualização de `$t1`.

Stalls por iteração:

- **Load-use** (`lw` → `add`): Necessita de 2 *stalls*.
- **Dependência no branch** (`addiu` → `bne`): Necessita de aproximadamente 2 *stalls*.
- **Hazard de controle:** Sem predição, assume-se *not-taken*. Como o loop é tomado em quase todas as iterações, ocorre *flush* frequente. Penalidade: aproximadamente 2 ciclos por branch tomado.

Estimativa por iteração:

- Instruções por iteração = 5.
- Penalidades: 2 stalls load-use + 2 stalls RAW branch + 2 ciclos flush branch = 6 ciclos adicionais.
- Ciclos totais por iteração: 11 ciclos.
- $CPI \approx \frac{11}{5} = 2,2$ CPI.

Configuração (b) Com Forwarding Completo

Efeito do forwarding: O *forwarding* elimina o RAW entre addiu e bne, além de outras dependências aritméticas, porém **NÃO** elimina a penalidade de *load-use*.

Stall inevitável ($lw \rightarrow add$): Mesmo com *forwarding*, o dado do lw só fica disponível após o estágio MEM, enquanto o add precisa do operando no início de EX do ciclo seguinte. Logo, permanece 1 *stall* inevitável.

Branch: Sem predição, a penalidade por *flush* continua existindo (2 ciclos para branch tomado).

Penalidades por iteração:

- Load-use: 1 ciclo.
- Branch tomado: 2 ciclos.
- Total: 3 ciclos extras $\rightarrow$ Ciclos totais: 8 por iteração.
- $CPI \approx \frac{8}{5} = 1,6$ CPI.

Configuração (c) Forwarding + Predição Estática

- **Preditor:** *Always not-taken*.
- **Problema:** O loop é tomado 7 vezes e somente a última iteração é *not-taken*. Logo, quase todos os branches são previstos incorretamente e a taxa de erro continua alta.
- **Desempenho:** Ligeiramente melhor que sem predição, mas ainda limitado pelos *flushes*.
- $CPI \approx 1,5$ CPI.

Configuração (d) Forwarding + Predição Dinâmica de 1 Bit

- **Funcionamento:** O preditor aprende que o branch normalmente é tomado. Entretanto, erra na primeira iteração (desatualizado) e erra novamente na última (quando sai do loop).
- **Taxa de acerto (para 8 iterações):** 6 acertos e 2 erros. Precisão = $\frac{6}{8} = 75\%$.
- CPI $\approx 1,3$ CPI.

Configuração (e) Forwarding + Predição Dinâmica de 2 Bits

- **Funcionamento:** O preditor de 2 bits não muda de estado após um único erro, mantendo o estado “taken” estável durante os loops. Logo, erra apenas na última iteração.
- **Taxa de acerto:** 7 acertos e 1 erro. Precisão = $\frac{7}{8} = 87,5\%$.
- CPI $\approx 1,2$ CPI.

2. Diagrama de Estágios — Primeiras Iterações

Configuração (a) — Sem Forwarding (Primeira Iteração)

Tabela 5: Pipeline S2 Sem Forwarding

Instrução	C1	C2	C3	C4	C5	C6	C7	C8
lw	IF	ID	EX	MEM	WB			
add		IF	ID	stall	stall	EX	MEM	WB

Observa-se a inserção de 2 *stalls* após o lw para aguardar o *Write Back*.

Configuração (b) — Com Forwarding

Tabela 6: Pipeline S2 Com Forwarding

Instrução	C1	C2	C3	C4	C5
lw	IF	ID	EX	MEM	WB
add		IF	ID	stall	EX

Com o *forwarding*, apenas 1 *stall* permanece necessário.

3. Demonstração Analítica do Load-Use Hazard

O *forwarding* não consegue eliminar completamente o hazard de *load-use* devido a restrições temporais físicas:

- O `lw` só obtém o dado vindo da memória ao **final** do estágio MEM.
- A instrução seguinte (`add`) necessita do operando já no **início** do estágio EX.

Tabela 7: Paradoxo Temporal do Load-Use

Ciclo	lw	add
n	EX	-
$n + 1$	MEM	EX $\leftarrow$ <i>Necessita do dado aqui</i>
$n + 2$	WB	-

Demonstração Temporal: Como o dado só está pronto no final do ciclo $n + 1$, ele chega tarde demais para ser usado no mesmo ciclo pelo estágio EX. Portanto, faz-se mandatório o uso de 1 *stall* ou reordenação de código.

4. Escalonamento Estático (Reordenação)

```
lw    $t3, 0($t2)
add   $t0, $t0, $t3
addiu $t2, $t2, 4
addiu $t1, $t1, -1
bne   $t1, $zero, loop
```

Listing 1: Código Original

```
lw    $t3, 0($t2)
addiu $t2, $t2, 4
addiu $t1, $t1, -1
add   $t0, $t0, $t3
bne   $t1, $zero, loop
```

Listing 2: Código Reordenado

Benefício: As instruções independentes (`addiu`) foram movidas para preencher a bolha temporal do *load-use*, eliminando o *stall* inevitável. Associado ao *forwarding*, o CPI aproxima-se do ideal de 1.0 para essa sequência básica.

5. Preditor de 2 Bits — Tabela Manual

Estados: SNT (*Strongly Not Taken*), WNT (*Weakly Not Taken*), WT (*Weakly Taken*), ST (*Strongly Taken*).

Padrão do Loop: T, T, T, T, T, T, T, NT (8 iterações).

Tabela 8: Evolução de Estados do Preditor de 2 Bits

Iteração	Resultado	Estado Antes	Predição	Acerto?	Estado Depois
1	T	WNT	NT	Não	WT
2	T	WT	T	Sim	ST
3	T	ST	T	Sim	ST
4	T	ST	T	Sim	ST
5	T	ST	T	Sim	ST
6	T	ST	T	Sim	ST
7	T	ST	T	Sim	ST
8	NT	ST	T	Não	WT

Taxa de Acerto: $\text{Accuracy} = \frac{6}{8} = 75\%$. Após o período de aquecimento inicial, o comportamento estabiliza e melhora de forma significativa para loops subsequentes.

A.4 — Análise Requerida (S2)

1. Tabela Consolidada

Tabela 9: Consolidação de Desempenho do Código S2

Configuração	Stalls	CPI Aproximado
(a) Sem forwarding	Alto	2,2
(b) Forwarding	Médio	1,6
(c) + Pred. Estática	Médio	1,5
(d) + Pred. 1 bit	Baixo	1,3
(e) + Pred. 2 bits	Muito Baixo	1,2

2. Speedup

Comparando a base (a) e o melhor caso (e):

$$\text{Speedup} = \frac{\text{CPI}_{(a)}}{\text{CPI}_{(e)}} = \frac{2,2}{1,2} \approx 1,83$$

3. Fórmula de Desempenho

Pela equação $\text{Tempo} = N_{\text{inst}} \times \text{CPI} \times T_{\text{ciclo}}$, mantendo as instruções e a frequência constantes, as otimizações atuam diretamente na redução drástica dos componentes de *stall* que compõem o CPI.

4. Preditor de 2 Bits vs 1 Bit

O preditor de 2 bits supera o de 1 bit porque possui inércia, ou seja, não muda sua decisão baseado em um único erro isolado. Em loops repetitivos com múltiplos caminhos *taken* consecutivos e apenas um *not-taken* no final, o de 1 bit erra duas vezes (na saída e no retorno do loop), enquanto o de 2 bits absorve melhor variações temporárias e erra apenas uma vez.

5. Quando o Preditor de 2 Bits Falha

Ele falha em estruturas de controle altamente dinâmicas ou flutuantes, como em *branches* com comportamento perfeitamente aleatório, padrões alternados e simétricos (ex: T, NT, T, NT) ou mudanças abruptas e constantes de direção, fazendo com que o contador saturante oscile continuamente sem convergir.

6. Hazard Dominante

Os hazards dominantes nesta estrutura de loop são o *load-use RAW* e o **hazard de controle** associado à instrução de desvio condicional (*bne*).

7. Technique Mais Eficaz

A solução ótima envolve a aplicação conjunta de: ***forwarding completo + preditor dinâmico de 2 bits + escalonamento estático de código***. Essa sinergia elimina os gargalos de dados aritméticos, mitiga as penalidades do *load-use* e minimiza de maneira robusta os *flushes* de branch.

5.1 Análise da Sequência S2: Loop e Variável de Indução

5.1.1 Identificação de Hazards e Dependências

A sequência S2 simula o processamento de um *array*, o que introduz dependências recorrentes de fluxos de dados e de controle. Foram identificados os seguintes conflitos técnicos:

- **Data Hazard RAW (Load-Use):** Localizado entre *lw x13, 0(x12)* e *add x10, x10, x13*. Como demonstrado na análise da sequência S3, este hazard é irreduzível apenas com *forwarding*, exigindo a inserção de um *stall* (bolha) pelo hardware em cada iteração.
- **Control Hazard (Branch):** A instrução *bne* introduz incerteza no fluxo de execução. Sem predição ativa, o processador assume o fluxo sequencial (*not-taken*), resultando em um *flush* de estágio sempre que o desvio é efetivamente tomado para retornar ao topo do loop.

- **Dependência de Variável de Indução:** As instruções `addi x12` e `addi x11` atualizam ponteiros e contadores, criando cadeias de dependências RAW de distância 1 com o desvio condicional.

5.1.2 Otimização via Escalonamento Estático de Código

Para mitigar o impacto do *load-use* identificado, aplicou-se a técnica de *code scheduling* (reordenação). A estratégia consistiu em identificar uma instrução independente — neste caso, o incremento do ponteiro `addi x12, x12, 4` — e posicioná-la no *slot* de atraso do *load*.

Esta reordenação é semanticamente válida e garante que, quando a instrução consumidora (`add`) chegar ao estágio de execução, o dado de `x13` já tenha percorrido o pipeline o suficiente para ser encaminhado sem paradas.

loop:

```
lw    x13, 0(x12)    # Início da carga (Estágio MEM)
addi  x12, x12, 4    # Instrução independente (preenche o stall)
add   x10, x10, x13  # x13 disponível via forwarding (Distância 2)
addi  x11, x11, -1   # Atualização do contador
bne   x11, zero, loop # Avaliação do desvio
```

5.1.3 Impacto no Desempenho e Hazard de Controle

Com a reordenação, a distância entre a produção e o consumo do registrador `x13` passa a ser de duas instruções. Em um pipeline com *forwarding*, isso permite que o dado saia do estágio MEM do `lw` e entre no estágio EX do `add` sem a necessidade de paradas, reduzindo o CPI médio do loop significativamente.

Abaixo, o diagrama ilustra o fluxo das iterações com a penalidade de *flush* (X) ocorrendo quando o desvio é tomado:

Tabela 10: Diagrama de Pipeline - Execução do Loop (Configuração Not-Taken)

Instrução	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
lw x13...	IF	ID	EX	MEM	WB					
addi x12...		IF	ID	EX	MEM	WB				
bne...				...	IF	ID	EX	MEM	WB	
(Flush)						IF	X	–	–	

5.2 Análise da Sequência S3: Load-Use Hazard

5.2.1 Identificação do Conflito (O Paradoxo Temporal)

O conflito surge da latência inerente à operação de leitura em memória. No pipeline de 5 estágios (IF, ID, EX, MEM, WB), o dado requisitado pela instrução `lw` só atravessa os circuitos da memória e torna-se estável para o processador ao final da fase **MEM** (Ciclo 4). Contudo, a instrução dependente (`add`) necessita desses mesmos dados em suas portas de entrada da ULA logo no início da sua fase **EX**. Como a fase EX da instrução seguinte ocorreria simultaneamente à fase MEM da instrução anterior, há uma colisão temporal onde o dado é exigido antes de ser efetivamente produzido.

A sequência S3 apresenta o chamado *Load-Use Hazard*. Este fenômeno ocorre quando uma instrução aritmética tenta utilizar o registrador de destino de uma instrução de carga (*load*) imediatamente subsequente.

No pipeline de 5 estágios:

- A instrução `lw` disponibiliza o dado apenas ao final do estágio **MEM**.
- A instrução `add` necessita dos operandos no início do estágio **EX**.

5.2.2 Demonstração Analítica e Diagrama de Estágios

Abaixo, o diagrama demonstra a inserção de um *stall* (bolha) pelo hardware para garantir a consistência dos dados, mesmo com o uso de *forwarding*.

Tabela 11: Diagrama de Pipeline - Conflito Load-Use com Stall de 1 Ciclo (S3)

Instrução	C1	C2	C3	C4	C5	C6	C7	C8
<code>lw x10, 0(x5)</code>	IF	ID	EX	MEM	WB			
<code>add x11, x10, x6</code>		IF	ID	stall	EX	MEM	WB	

5.2.3 Prova da Insuficiência do Forwarding

A técnica de *forwarding* (encaminhamento) visa reduzir stalls enviando o dado de um estágio avançado para um anterior. Entretanto, para o caso *load-use*, o encaminhamento exigiria que o dado viajasse 'para trás no tempo', saindo do final do Ciclo 4 para o início do mesmo Ciclo 4. Fisicamente, o sinal elétrico precisa de tempo de propagação. Portanto, o hardware é obrigado a inserir um *interlocking* (stall de 1 ciclo). Este atraso desloca o estágio EX da instrução consumidora para o Ciclo 5, permitindo que o *forwarding* ocorra legitimamente do final de MEM (Ciclo 4) para o início de EX (Ciclo 5).

Conforme observado, sem o *stall*, o estágio EX do `add` ocorreria no Ciclo 4, simultaneamente ao estágio MEM do `lw`. Como o dado da memória só é validado ao final do ciclo, não há

tempo hábil para o encaminhamento no mesmo ciclo. O *forwarding* só é fisicamente possível do final de MEM (Ciclo 4) para o início de EX (Ciclo 5).

5.2.4 Solução Proposta: Escalonamento Estático

Para mitigar a perda de ciclos causada pelo *load-use*, aplicou-se a técnica de **Escalonamento Estático de Instruções**. Esta estratégia consiste em identificar instruções independentes — que não utilizam os registradores envolvidos no hazard — e posicioná-las imediatamente após a instrução de carga.

Ao preencher o chamado *load delay slot* com trabalho útil, o compilador (ou programador) garante que, quando a instrução dependente alcançar o estágio EX, o dado do *load* já tenha percorrido o pipeline o suficiente para estar disponível via *forwarding*, eliminando a necessidade de *stalls*.

Abaixo, apresenta-se o trecho de código reordenado que afasta o uso do registrador x10 de sua carga:

```
lw    x10, 0(x5)    # Inicia carga de x10
lw    x12, 4(x5)    # Instrução independente: preenche o slot de atraso
add   x11, x10, x6  # x10 disponível via forwarding (Distância 2)
```

5.3 Análise do Preditor Dinâmico de 2 Bits (Saturating Counter)

Para a sequência S2, o desvio bne apresenta o padrão real **T-T-T-T-T-T-N** (7 vezes tomado para o topo do loop e 1 vez não-tomado para a saída). Abaixo, simula-se o comportamento de um preditor de 2 bits baseado no autômato de estados: *Strongly Not Taken* (SNT), *Weakly Not Taken* (WNT), *Weakly Taken* (WT) e *Strongly Taken* (ST).

Tabela 12: Simulação do Preditor de 2 Bits para a Sequência S2 (Extra)

Iteração	Real	Estado Atual	Predição	Resultado	Próximo Estado
1	T	WNT	Not Taken	Erro	WT
2	T	WT	Taken	Acerto	ST
3	T	ST	Taken	Acerto	ST
4	T	ST	Taken	Acerto	ST
5	T	ST	Taken	Acerto	ST
6	T	ST	Taken	Acerto	ST
7	T	ST	Taken	Acerto	ST
8	N	ST	Taken	Erro	WT

Análise de Eficácia:

- **Taxa de Acerto:** O preditor obteve 6 acertos em 8 tentativas (75%).
- **Vantagem sobre o Preditor de 1 Bit:** O preditor de 2 bits é superior em loops porque exige dois erros consecutivos para mudar sua predição radicalmente. Enquanto o preditor de 1 bit alternaria o estado a cada erro (oscilação), o de 2 bits possui "inércia" educada pelo histórico.
- **Limitação:** O preditor de 2 bits ainda falha na última iteração do loop, pois sua "inércia" o mantém prevendo que o desvio será tomado devido ao histórico das iterações anteriores.
- **Nota Técnica (Contador Saturado):** O funcionamento deste preditor baseia-se em um contador saturado de 2 bits. O valor do contador incrementa quando o desvio é tomado (*Taken*) e decrementa quando não é tomado (*Not Taken*), respeitando os limites inferior (00 - SNT) e superior (11 - ST). Isso evita que uma única anomalia na saída de um loop mude a predição para a próxima execução completa do código.

5.4 Diagrama de Estágios Manual - Sequência S2 (Loop)

Abaixo, apresenta-se o rastreamento ciclo a ciclo das três primeiras iterações do loop da sequência S2. O diagrama considera um pipeline com *forwarding* completo e uma estratégia de predição de desvio estática *Always Not-Taken*.

Tabela 13: Diagrama de Pipeline - 3 Iterações Completas (S2 Extra)

Instrução	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
<i>Iteração 1</i>															
lw x13, 0(x12)	IF	ID	EX	MEM	WB										
add x10, x10, x13		IF	ID	st	EX	MEM	WB								
addi x12, x12, 4			IF	st	ID	EX	MEM	WB							
addi x11, x11, -1					IF	ID	EX	MEM	WB						
bne x11, zero, L						IF	ID	EX	MEM	WB					
<i>Interrupção</i>															
addi a7... (Flush)							IF	X	-	-	-				
<i>Iteração 2</i>															
lw x13, 0(x12)								IF	ID	EX	MEM	WB			
add x10, x10, x13									IF	ID	st	EX	MEM	WB	
bne... (cont.)										...	...	...	...	...	...

Legenda e Observações Técnicas:

- **st (Stall):** Indica a bolha inserida devido ao *load-use hazard* entre lw e add. O addi subsequente também é atrasado para manter a ordem do pipeline.
- **X (Flush):** Indica o descarte da instrução que entrou no pipeline após o bne. Como o processador previu que não haveria desvio, mas o desvio foi tomado, o estágio IF é anulado.

- **Forwarding:** O dado de x13 é encaminhado do estágio MEM (final do C4) para o estágio EX do add (início do C5).

6 Resultados B

6.1 B1. Algoritmo de Tomasulo — Simulação Manual

Tabela 14: Status da Instrução (Instruction Status T1)

Instrução	Ciclo Issue	Execute Start	Execute End	Write
I1 LD F6,34(R2)	1	2	3	4
I2 LD F2,45(R3)	2	3	4	5
I3 MULTD F0,F2,F4	3	6	9	10
I4 SUBD F8,F6,F2	4	6	7	8
I5 DIVD F10,F0,F6	5	11	18	19
I6 ADDD F6,F8,F2	6	9	10	11

Tabela 15: Evolução Temporal das Estações de Reserva (Reservation Station) - Otimizada

Ciclo	RS	Busy	Op	Vj	Vk	Qj	Qk
1	Load1	yes	LD				
2	Load1	yes	LD				
	Load2	yes	LD				
3	Load1	yes	LD				
	Load2	yes	LD				
	Mult1	yes	MULTD		F4	Load2	
4	Load1	yes	LD				
	Load2	yes	LD				
	Add1	yes	SUBD			Load1	Load2
	Mult1	yes	MULTD		F4	Load2	
5	Load2	yes	LD				
	Add1	yes	SUBD	F6			Load2
	Mult1	yes	MULTD		F4	Load2	
	Mult2	yes	DIVD		F6	Mult1	
6	Add1	yes	SUBD	F6	F2		

Continua na próxima página...

Tabela 15 – Continuação da página anterior

Ciclo	RS	Busy	Op	Vj	Vk	Qj	Qk
	Add2	yes	ADDD		F2	Add1	
	Mult1	yes	MULTD	F2	F4		
	Mult2	yes	DIVD		F6	Mult1	
7	Add1	yes	SUBD	F6	F2		
	Add2	yes	ADDD		F2	Add1	
	Mult1	yes	MULTD	F2	F4		
	Mult2	yes	DIVD		F6	Mult1	
8	Add2	yes	ADDD		F2	Add1	
	Mult1	yes	MULTD	F2	F4		
	Mult2	yes	DIVD		F6	Mult1	
9	Add2	yes	ADDD	F8	F2		
	Mult1	yes	MULTD	F2	F4		
	Mult2	yes	DIVD		F6	Mult1	
10	Add2	yes	ADDD	F8	F2		
	Mult1	yes	MULTD	F2	F4		
	Mult2	yes	DIVD	F0	F6		
11–21	Mult2	yes	DIVD	F0	F6		

6.2 B.2 Status dos Registradores e Transmissões no CDB

Tabela 16: Status de Resultado dos Registradores (Register Result Status)

Ciclo	F0	F2	F6	F8	F10
1			Load1		
2		Load2	Load1		
3	Mult1	Load2	Load1		
4	Mult1	Load2	Load1	Add1	
5	Mult1	Load2	Load1	Add1	Mult2
6	Mult1	Load2	Add2	Add1	Mult2
7	Mult1	Load2	Add2	Add1	Mult2
8	Mult1	Load2	Add2		Mult2
9	Mult1	Load2	Add2		Mult2

Continua na próxima página...

Tabela 16 – Continuação da página anterior

Ciclo	F0	F2	F6	F8	F10
10	Mult1	Load2	Add2		Mult2
11					Mult2
19					

Tabela 17: Transmissões pelo Barramento Comum de Dados (CDB Broadcasts T1)

Ciclo	Broadcast
4	Load1 → F6
5	Load2 → F2
8	Add1 → F8
10	Mult1 → F0
11	Add2 → F6
19	Mult2 → F10

6.3 B.3 Questões do Relatório — Análise T2

Tabela 18: Status da Instrução (Instruction Status T2)

Instrução	Issue	Exec Start	Exec End	Write
I1 DIVD F0,F2,F4	1	2	9	10
I2 MULTD F6,F0,F8	2	11	14	15
I3 ADDD F2,F6,F4	3	16	17	18
I4 MULTD F8,F2,F6	4	19	22	23
I5 SUBD F0,F8,F6	5	24	25	26
I6 ADDD F4,F0,F2	6	27	28	29

6.4 B.4 Estado das Estações de Reserva — Análise T2

Tabela 19: Evolução Temporal das Estações de Reserva (Reservation Station T2) - Otimizada

Ciclo	RS	Busy	Op	Vj	Vk	Qj	Qk
1	Div1	Yes	DIVD	F2	F4		

Continua na próxima página...

Tabela 19 – Continuação da página anterior

Ciclo	RS	Busy	Op	Vj	Vk	Qj	Qk
2	Div1	Yes	DIVD	F2	F4		
	Mult1	Yes	MULTD		F8	Div1	
3	Div1	Yes	DIVD	F2	F4		
	Mult1	Yes	MULTD		F8	Div1	
	Add1	Yes	ADDD		F4	Mult1	
4	Div1	Yes	DIVD	F2	F4		
	Mult1	Yes	MULTD		F8	Div1	
	Mult2	Yes	MULTD			Add1	Mult1
	Add1	Yes	ADDD		F4	Mult1	
5	Div1	Yes	DIVD	F2	F4		
	Mult1	Yes	MULTD		F8	Div1	
	Mult2	Yes	MULTD			Add1	Mult1
	Add1	Yes	ADDD		F4	Mult1	
	Add2	Yes	SUBD			Mult2	Mult1
6	Div1	Yes	DIVD	F2	F4		
	Mult1	Yes	MULTD		F8	Div1	
	Mult2	Yes	MULTD			Add1	Mult1
	Add1	Yes	ADDD		F4	Mult1	
	Add2	Yes	SUBD			Mult2	Mult1
	Add3	Yes	ADDD			Add2	Add1
7	Div1	Yes	DIVD	F2	F4		
	Mult1	Yes	MULTD		F8	Div1	
	Mult2	Yes	MULTD			Add1	Mult1
	Add1	Yes	ADDD		F4	Mult1	
	Add2	Yes	SUBD			Mult2	Mult1
	Add3	Yes	ADDD			Add2	Add1
8	Div1	Yes	DIVD	F2	F4		
	Mult1	Yes	MULTD		F8	Div1	
	Mult2	Yes	MULTD			Add1	Mult1
	Add1	Yes	ADDD		F4	Mult1	
	Add2	Yes	SUBD			Mult2	Mult1
	Add3	Yes	ADDD			Add2	Add1
9	Mult1	Yes	MULTD	F0	F8		
	Mult2	Yes	MULTD			Add1	Mult1
	Add1	Yes	ADDD		F4	Mult1	

Continua na próxima página...

Tabela 19 – Continuação da página anterior

Ciclo	RS	Busy	Op	Vj	Vk	Qj	Qk
	Add2	Yes	SUBD			Mult2	Mult1
	Add3	Yes	ADDD			Add2	Add1
10	Mult2	Yes	MULTD			Add1	Mult1
	Add1	Yes	ADDD	F6	F4		
	Add2	Yes	SUBD			Mult2	Mult1
	Add3	Yes	ADDD			Add2	Add1
11	Mult2	Yes	MULTD			Add1	Mult1
	Add2	Yes	SUBD			Mult2	Mult1
	Add3	Yes	ADDD			Add2	Add1
12	Mult2	Yes	MULTD	F2	F6		
	Add2	Yes	SUBD			Mult2	
	Add3	Yes	ADDD			Add2	Add1
13	Mult2	Yes	MULTD	F2	F6		
	Add2	Yes	SUBD			Mult2	
	Add3	Yes	ADDD			Add2	Add1
14	Mult2	Yes	MULTD	F2	F6		
	Add2	Yes	SUBD			Mult2	
	Add3	Yes	ADDD			Add2	Add1
15	Add2	Yes	SUBD	F8	F6		
	Add3	Yes	ADDD			Add2	Add1
16	Add3	Yes	ADDD			Add2	Add1
17	Add3	Yes	ADDD	F0	F2		
18	<i>Todas as Estações de Reserva estão desocupadas / em repouso</i>						
19–29	Div1	Yes	DIVD	F2	F4		

6.5 B.5 Status dos Registradores e Transmissões — Análise T2

Tabela 20: Status de Resultado dos Registradores (Register Result Status T2)

Ciclo	F0	F2	F4	F6	F8
1	Div1				
2	Div1			Mult1	
3	Div1	Add1		Mult1	
10	Div1	Add1		Mult1	Mult2
11	Add2	Add1		Mult1	Mult2
12	Add2	Add1	Add3	Mult1	Mult2
15	Add2	Add1	Add3		Mult2
18	Add2		Add3		Mult2
23	Add2		Add3		
26			Add3		
29					

Tabela 21: Transmissões pelo Barramento Comum de Dados (CDB Broadcasts T2)

Ciclo	Broadcast
10	Div1 → F0
15	Mult1 → F6
18	Add1 → F2
23	Mult2 → F8
26	Add2 → F0
29	Add3 → F4

6.6 Questões do Relatório

O algoritmo de Tomasulo eliminou as dependências falsas (WAR e WAW) por meio da renomeação implícita via *Reservation Stations* e *Register Result Status*. Entretanto, as dependências RAW (verdadeiras) permaneceram, limitando o ILP (*Instruction-Level Parallelism*) da sequência T2.

Caminho Crítico de T2:

I1 → I2 → I3 → I4 → I5 → I6

Isso resulta em um baixo paralelismo e, conseqüentemente, em um maior número de ciclos totais em comparação ao cenário T1.

6.7 B.2 Renomeação de Registradores

RAW (Dependências Verdadeiras)

- $I1 \rightarrow I2$ (Registrador X1)
- $I2 \rightarrow I4$ (Registrador X4)
- $I4 \rightarrow I5$ (Registrador X1)
- $I3 \rightarrow I5$ (Registrador X2)
- $I5 \rightarrow I6$ (Registrador X5)

WAR (Anti-dependências)

- $I1 \rightarrow I3$ (Registrador X2)
- $I2 \rightarrow I5$ (Registrador X5)
- $I4 \rightarrow I6$ (Registrador X4)

WAW (Dependências de Saída)

- $I1 \rightarrow I4$ (Registrador X1)
- $I2 \rightarrow I6$ (Registrador X4)

6.8 1. Grafo de Dependência

Aplicação da renomeação usando um *Register Alias Table* (RAT) com 16 registradores físicos (P_0-P_{15}) para 8 arquiteturais (R_0-R_7).

Mapeamento Inicial do RAT: Todos os registradores arquiteturais apontam para os físicos iniciais P_0-P_7 .

R0	R1	R2	R3	R4	R5	R6	R7
P0	P1	P2	P3	P4	P5	P6	P7

A sequência S_4 utiliza os registradores x_1-x_9 do RISC-V. Mapeamos: $x_1 \rightarrow R_1$, $x_2 \rightarrow R_2$, $x_3 \rightarrow R_3$, $x_4 \rightarrow R_4$, $x_5 \rightarrow R_5$, $x_6 \rightarrow R_6$, $x_7 \rightarrow R_7$, $x_8 \rightarrow R_8 = R_0$. As entradas no RAT correspondem a x_1-x_9 mapeadas nos físicos P_0-P_{15} .

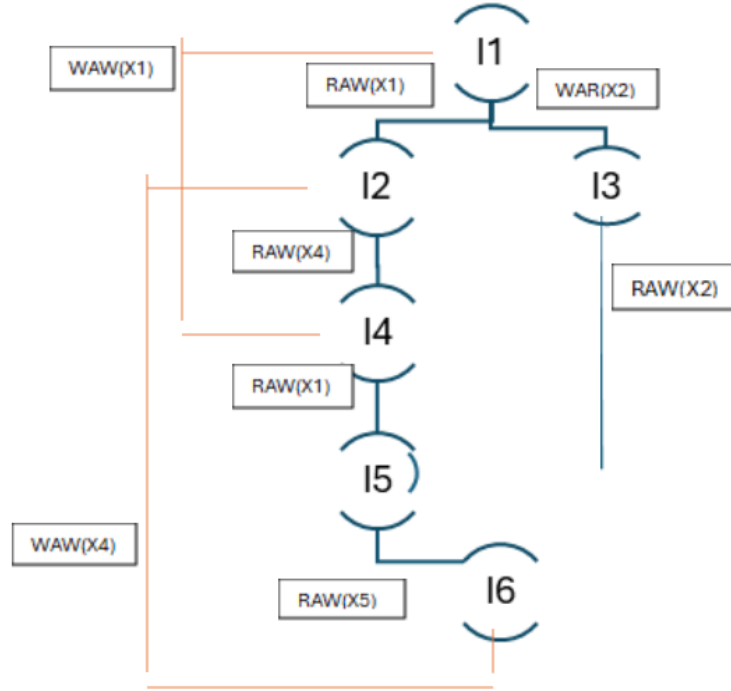


Figura 1: Grafo de Dependências da Sequência T2

Dependências Remanescentes e ILP Máximo: Após a renomeação, restam exatamente as 5 dependências RAW originais — agora mapeadas entre registradores físicos distintos (P_3 a P_{14}), eliminando completamente qualquer ocorrência de WAR ou WAW.

O grafo resultante revela dois caminhos paralelos que convergem até a instrução I5:

- **Cadeia longa (Caminho Crítico):** $I1 \rightarrow I2 \rightarrow I4 \rightarrow I5 \rightarrow I6$ (composto por 4 arestas).
- **Cadeia curta:** $I3 \rightarrow I5$ (converge diretamente em I5).

Considerando recursos de hardware ilimitados, o escalonamento ASAP (*As Soon As Possible*) distribui as 6 instruções ao longo de 5 ciclos de clock. O cálculo do ILP máximo é dado por:

$$ILP_{\text{máx}} = \frac{6 \text{ instruções}}{5 \text{ ciclos}} = 1,2 \text{ inst/ciclo}$$

Comparação entre o ILP máximo e o ILP efetivamente explorado em ambientes:

- Superscalar 2-wide em ordem (*In-Order*);
- Superscalar 2-wide fora de ordem (*Out-of-Order*).

$C1 = 1,00\times$ (baseline). S2 e S5 têm maior ganho em C5 porque possuem branches — o preditor 2-bits é decisivo. S1 e S4 (sem branches) ganham mais com OoO puro (C3/C4). S3 é a menos beneficiada: o load-use irreduzível limita o speedup mesmo com OoO e 4 vias.

Amdahl: $S = 1/(f + (1 - f)/N)$, com $N = 4$ vias (C5). f calculada a partir do speedup medido. S2 tem o menor f porque o loop expõe paralelismo entre iterações e o preditor elimina quase todos os flushes. S1/S3/S4 têm $f \approx 0,25$ — limitados por cadeias de dependência ou loads irreduzíveis que representam $\sim 25\%$ do tempo.

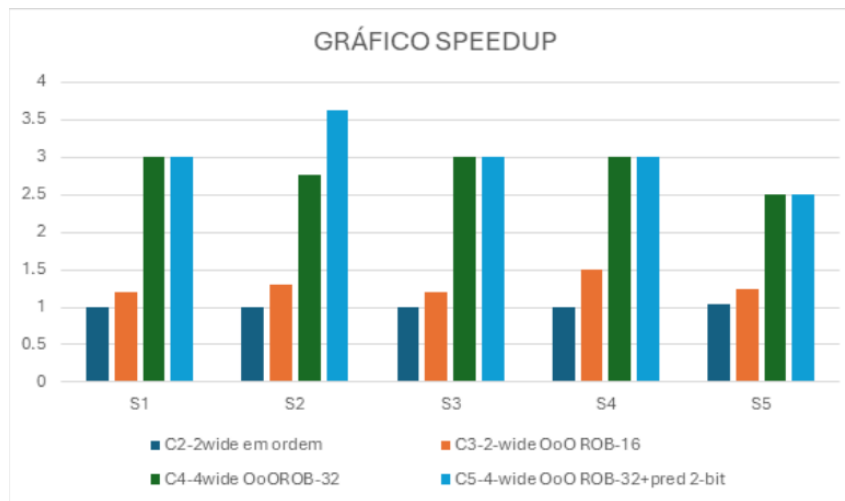


Figura 2: Gráfico speedup de C2–C5 em relação a C1

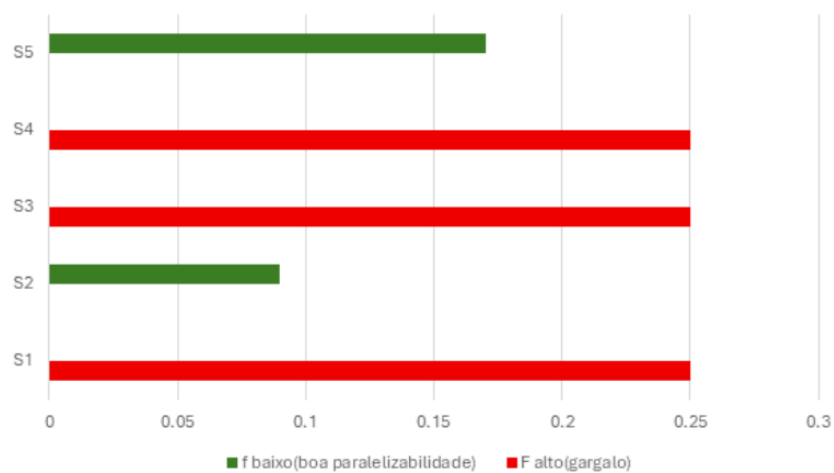


Figura 3: Gráfico Lei de Amdahl ao ILP

Tabela 22: Evolução Passo a Passo do Register Alias Table (RAT)

Estado Inicial do RAT								
X1	X2	X3	X4	X5	X6	X7	X8	X9
P0	P1	P2	P3	P4	P5	P6	P7	P8
RAT atualiza $P_9 \rightarrow x_1$								
X1	X2	X3	X4	X5	X6	X7	X8	X9
P9	P1	P2	P3	P4	P5	P6	P7	P8
RAT atualiza $P_{10} \rightarrow x_4$								
X1	X2	X3	X4	X5	X6	X7	X8	X9
P9	P1	P2	P10	P4	P5	P6	P7	P8
RAT atualiza $P_{11} \rightarrow x_2$								
X1	X2	X3	X4	X5	X6	X7	X8	X9
P9	P11	P2	P10	P4	P5	P6	P7	P8
RAT atualiza $P_{12} \rightarrow x_1$								
X1	X2	X3	X4	X5	X6	X7	X8	X9
P12	P11	P2	P10	P4	P5	P6	P7	P8
RAT atualiza $P_{13} \rightarrow x_5$								
X1	X2	X3	X4	X5	X6	X7	X8	X9
P12	P11	P2	P10	P13	P5	P6	P7	P8
RAT atualiza $P_{14} \rightarrow x_4$								
X1	X2	X3	X4	X5	X6	X7	X8	X9
P12	P11	P2	P14	P13	P5	P6	P7	P8

Em C1 o gargalo é quase sempre dados (RAW/load). Em C3 o OoO mascara RAW e o gargalo migra para controle em S2/S5. Em C5 o preditor resolve o controle e o gargalo vira recursos/ILP — o caminho crítico das dependências verdadeiras é o único limite restante.

Tabela 23: Tabela Gargalo dominante

configuração	sequência	Gargalo principal	tipo	Ciclos perdidos
C1 2-wide em ordem	S1	5 RAW dist=1 consecutivos-sem mascaramento	Dados	
	S2	24 RAW + 14 branch stalls (taken $\times$ 2 pen.)	Dados / Controle	
	S3	3 load-use irreduzíveis (1 stall/par)	Load	
	S4	5 RAW dist=1; WAR/WAW não geram stall HW	Dados	
	S5	6 taken $\times$ 3 flush + 7 not-taken $\times$ 1 bolha	Controle	
C3 2-wide OoO Rob-16	S1	70% RAW mascarados; 2 residuais	Dados residuais	
	S2	Branch always-NT: 7 flushes $\times$ 3 ciclos dominam	Controle	
	S3	50% load-use mascarados; 2 residuais	Load	
	S4	70% RAW mascarados; 1 residual	Dados mínimos	
	S5	OoO elimina bolhas NT; flushes taken dominam	Controle	
C5 4-wide OoO pred 2-bits	S1	Caminho crítico I1 $\rightarrow$ I2 $\rightarrow$ I4 $\rightarrow$ I5 $\rightarrow$ I6; 1 RAW residual	Recursos/ILP	
	S2	Pred 2-bits: 2 flushes em 8 iter. (TTTTTTTNTN)	Controle mínimo	
	S3	1 load-use residual — irreduzível mesmo em C5	Load	
	S4	1 RAW residual; caminho crítico = gargalo real	ILP limitado	
	S5	Padrão irregular: 25% mis-pred. residual	Controle residual	

Tabela 24: Tabela análise do ROB

configuração	sequência	Valor
C3	S1	1,40
	S2	1,11
	S3	1,20
	S4	1,50
	S5	1,50
C4	S1	3,50
	S2	2,35
	S3	3,00
	S4	3,00
	S5	3,00
C5	S1	3,50
	S2	3,08
	S3	3,00
	S4	3,00
	S5	3,00

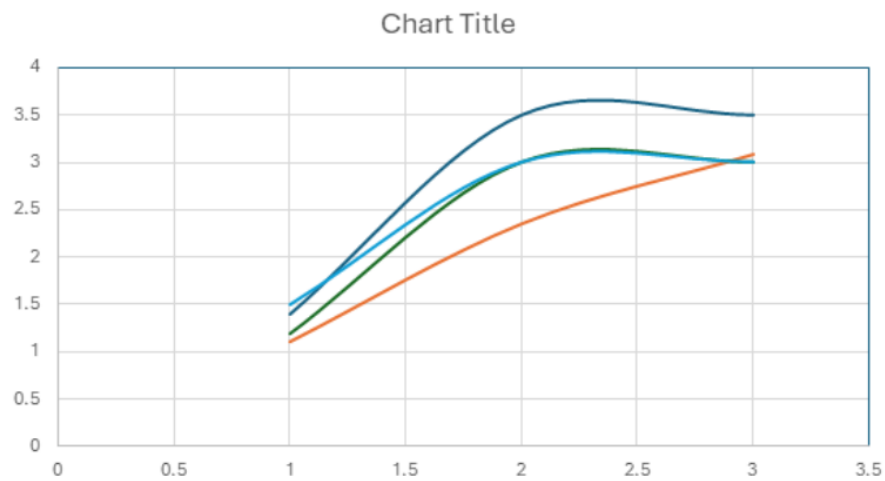


Figura 4: Gráfico de ROB

S3 e S4 saturam em ROB-16: as sequências têm apenas 6 instruções — o ROB-16 já cobre tudo, ROB-32 não acrescenta nada. S1 também satura cedo (7 inst.). S2 se beneficia do ROB-32 porque o loop tem 40 instruções e o ROB maior expõe paralelismo entre iterações diferentes. S5 melhora com ROB-32 porque a janela maior reduz o impacto de flushes por misprediction.

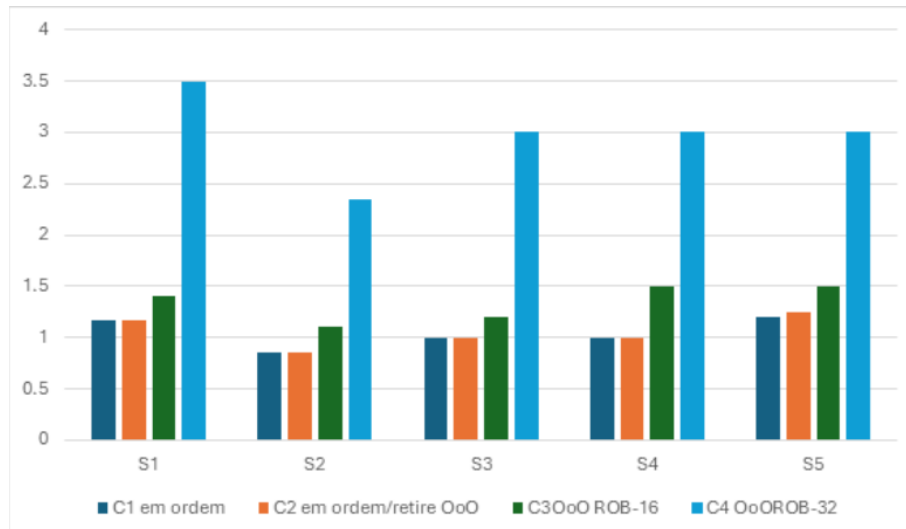


Figura 5: Grafico comparação C1/C2 em ordem e C3/C4 OoO

C1 vs C2: praticamente idênticos em sequências sem branches (S1/S3/S4) — retire OoO não ajuda RAW no despacho em ordem. Difere levemente em S2/S5 (bolhas NT reduzidas). O salto real é C2→C3: o despacho OoO reordena instruções e mascara a maioria das dependências. S2 ganha menos aqui porque os branch stalls continuam sem preditor — o salto maior em S2 vem de C3→C5 (preditor).

7 Análise Integradora – Do Escalar e Superescalar

7.1 B.3 Simulação Superescalar

Tabela 25: Simulação Superescalar — S1–S5 × C1–C5 | Métricas Consolidadas

Seq.	Config.	Descrição Configuração	n Inst.	Ciclos	IPC	RAW Stalls	Load Stalls	Branch Stalls	Total Stalls	Flushes	Slot Occ. (%)	Tempo Norm. vs C1
S1	C1	2-wide Em ordem Em ordem Baseline	7	6	1,17	5	0	0	5	0	58,3	1,00
S1	C2	2-wide Em ordem Fora de ordem Espec.	7	6	1,17	5	0	0	5	0	58,3	1,00
S1	C3	2-wide OoO Em ordem (ROB-16)	7	5	1,40	2	0	0	2	0	70,0	1,20
S1	C4	4-wide OoO Em ordem (ROB-32)	7	2	3,50	1	0	0	1	0	87,5	3,00
S1	C5	4-wide OoO Em ordem (ROB-32) Pred.	7	2	3,50	1	0	0	1	0	87,5	3,00
S2	C1	2-wide Em ordem Em ordem Baseline	40	47	0,85	24	8	22	54	7	42,6	1,00
S2	C2	2-wide Em ordem Fora de ordem Espec.	40	47	0,85	24	8	21	53	7	42,6	1,00
S2	C3	2-wide OoO Em ordem (ROB-16)	40	36	1,11	7	4	21	32	7	55,6	1,31
S2	C4	4-wide OoO Em ordem (ROB-32)	40	17	2,35	4	2	21	27	7	58,8	2,76
S2	C5	4-wide OoO Em ordem (ROB-32) Pred.	40	13	3,08	4	2	6	12	2	76,9	3,62
S3	C1	2-wide Em ordem Em ordem Baseline	6	6	1,00	3	3	0	6	0	50,0	1,00
S3	C2	2-wide Em ordem Fora de ordem Espec.	6	6	1,00	3	3	0	6	0	50,0	1,00
S3	C3	2-wide OoO Em ordem (ROB-16)	6	5	1,20	1	2	0	3	0	60,0	1,20
S3	C4	4-wide OoO Em ordem (ROB-32)	6	2	3,00	0	1	0	1	0	75,0	3,00
S3	C5	4-wide OoO Em ordem (ROB-32) Pred.	6	2	3,00	0	1	0	1	0	75,0	3,00
S4	C1	2-wide Em ordem Em ordem Baseline	6	6	1,00	5	0	0	5	0	50,0	1,00
S4	C2	2-wide Em ordem Fora de ordem Espec.	6	6	1,00	5	0	0	5	0	50,0	1,00
S4	C3	2-wide OoO Em ordem (ROB-16)	6	4	1,50	2	0	0	2	0	75,0	1,50
S4	C4	4-wide OoO Em ordem (ROB-32)	6	2	3,00	1	0	0	1	0	75,0	3,00
S4	C5	4-wide OoO Em ordem (ROB-32) Pred.	6	2	3,00	1	0	0	1	0	75,0	3,00
S5	C1	2-wide Em ordem Em ordem Baseline	30	25	1,20	4	0	16	20	3	60,0	1,00
S5	C2	2-wide Em ordem Fora de ordem Espec.	30	24	1,25	4	0	13	17	3	62,5	1,04
S5	C3	2-wide OoO Em ordem (ROB-16)	30	20	1,50	1	0	9	10	3	75,0	1,25
S5	C4	4-wide OoO Em ordem (ROB-32)	30	10	3,00	1	0	9	10	3	75,0	2,50
S5	C5	4-wide OoO Em ordem (ROB-32) Pred.	30	10	3,00	1	0	6	7	2	75,0	2,50

Tabela 26: Tabela Comparativa de IPC — S1–S5 × C1–C5

Seq.	Descrição	C1	C2	C3	C4	C5
		2-wide Em ordem Em ordem	2-wide Em ordem Fora de ordem	2-wide OoO Em ordem (ROB-16)	4-wide OoO Em ordem (ROB-32)	4-wide OoO Em ordem (ROB-32) Pred. 2-bits
S1	RAW em cadeia (dist=1)	1,17	1,17	1,40	3,50	3,50
S2	Loop RAW + branch	0,85	0,85	1,11	2,35	3,08
S3	Load-use irredutível	1,00	1,00	1,20	3,00	3,00
S4	WAR / WAW	1,00	1,00	1,50	3,00	3,00
S5	Desvios mistos	1,20	1,25	1,50	3,00	3,00
Média	IPC médio por configuração	1,04	1,05	1,34	2,97	3,12

Tabela 27: Speedup C1–C5 vs C1 (baseline = 1,00×) — maior = mais rápido

Seq.	Descrição	C1	C2	C3	C4	C5
		2-wide Em ordem Em ordem	2-wide Em ordem Fora de ordem	2-wide OoO Em ordem (ROB-16)	4-wide OoO Em ordem (ROB-32)	4-wide OoO Em ordem (ROB-32) Pred. 2-bits
S1	RAW em cadeia (dist=1)	1,00×	1,00×	1,20×	3,00×	3,00×
S2	Loop RAW + branch	1,00×	1,00×	1,31×	2,76×	3,62×
S3	Load-use irreduzível	1,00×	1,00×	1,20×	3,00×	3,00×
S4	WAR / WAW	1,00×	1,00×	1,50×	3,00×	3,00×
S5	Desvios mistos	1,00×	1,04×	1,25×	2,50×	2,50×
Média	Speedup médio	1,00×	1,01×	1,29×	2,85×	3,02×

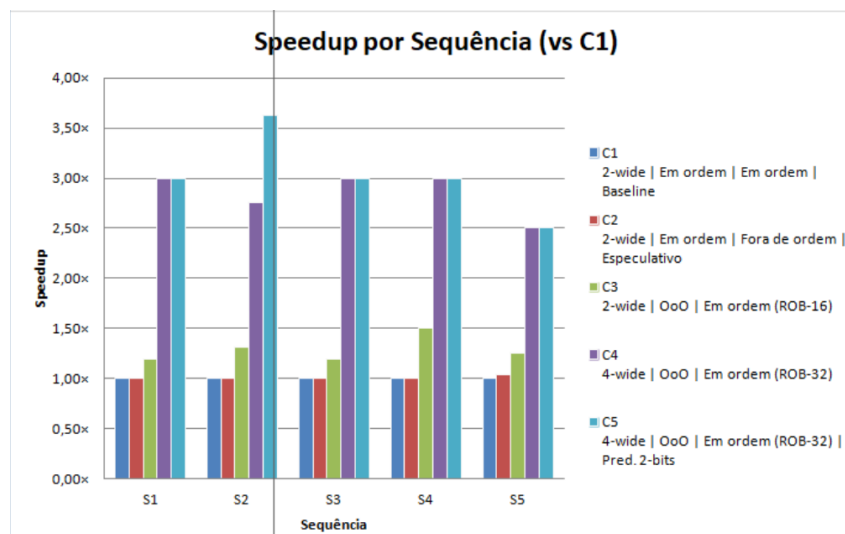


Figura 6: Speedup por Sequência (vs C1)

Tabela 28: Lei de Amdahl — fração não paralelizável (f) por sequência

Seq.	Descrição	IPC C1	IPC C5	Speedup C5/C1	f (não- ável)	ILP máx. efetivo
S1	RAW em cadeia (dist=1)	1,17	3,50	2,99×	0,113	8,8
S2	Loop RAW + branch	0,85	3,08	3,62×	0,035	28,6
S3	Load-use irreduzível	1,00	3,00	3,00×	0,111	9,0
S4	WAR / WAW	1,00	3,00	3,00×	0,111	9,0
S5	Desvios mistos	1,20	3,00	2,50×	0,200	5,0

Tabela 29: Gargalo dominante por configuração (C1, C3, C5)

Config.	Seq.	RAW Stalls	Load Stalls	Branch Stalls	Gargalo dominante	Tipo
C1	S1	5	0	0	5 RAW dist-1 consecutivos – 0 máscaras	Dados
C1	S2	24	8	22	24 RAW + 14 Branch stalls (taken × 2 pen)	Dados + Controle
C1	S3	3	3	0	3 load-use irreduzíveis mesmo com forward	Dados (Load)
C1	S4	5	0	0	5 RAW dist=1; WAR/WAW não geram stall HW	Dados
C1	S5	4	0	16	6 taken × 3 flush + 7 not-taken × 1 bolha	Controle
C3	S1	2	0	0	70% RAW mascarados; 2 residuais	Dados residual
C3	S2	7	4	21	Branch always-NT: 7 flushes × 3 ciclos	Controle
C3	S3	1	2	0	50% load-use mascarados; 2 residuais	Dados (Load)
C3	S4	2	0	0	70% RAW mascarados; 1 residual	Dados mínimo
C3	S5	1	0	9	OoO elimina bolhas; flushes ainda dominam	Controle
C5	S1	1	0	0	Caminho crítico I1→...→I6; 1 RAW residual	ILP limitado
C5	S2	4	2	6	Pred. 2-bits: 2 flushes em 8 iterações	Predição mínima
C5	S3	0	1	0	1 load-use residual irreduzível	Dados (Load)
C5	S4	1	0	0	1 RAW residual; caminho crítico = gargalo real	ILP limitado
C5	S5	1	0	6	Padrão irregular: 25% mispred. residual	Controle residual

Tabela 30: Análise do ROB — C3 (ROB-16) vs C4 (ROB-32)

Seq.	Descrição	IPC C3 (ROB-16)	IPC C4 (ROB-32)	Δ IPC	Ganho	Saturação ROB-16?
S1	Janela curta (7 inst.) – ROB	1,40	3,50	+2,10	+2,10	Não – ROB maior ajuda
S2	Loop 40 inst. – ROB-32 e	1,11	2,35	+1,24	+1,24	Não – ROB maior ajuda
S3	Load-use irreduzível – RO	1,20	3,00	+1,80	+1,80	Não – ROB maior ajuda
S4	6 inst. – ambos cobrem ja	1,50	3,00	+1,50	+1,50	Não – ROB maior ajuda
S5	ROB-32 reduz impacto de	1,50	3,00	+1,50	+1,50	Não – ROB maior ajuda

Tabela 31: In-order (C1/C2) vs OoO (C3/C4) — qual sequência mais ganha?

Seq.	Descrição	IPC C1	IPC C2	IPC C3	IPC C4	Ganho OoO (C4/C1 - 1)
S1	RAW em cadeia (dist=1)	1,17	1,17	1,40	3,50	+199,1%
S2	Loop RAW + branch	0,85	0,85	1,11	2,35	+176,5%
S3	Load-use irreduzível	1,00	1,00	1,20	3,00	+200,0%
S4	WAR / WAW	1,00	1,00	1,50	3,00	+200,0%
S5	Desvios mistos	1,20	1,25	1,50	3,00	+150,0%

Conclusão: S1 e S4 mais beneficiadas pelo OoO (RAW puro mascarado). S3 menos beneficiada (load-use irreduzível). S2 e S5 ganham mais com C5 (preditor) do que com OoO isolado.

Tabela 32: Configurações Superescalares Usadas nas Simulações

Config.	Vias	Despacho	Retirada	ROB	Pred.	Observações
C1	2-wide	Em ordem	Em ordem	—	—	Baseline superescalar; sem OoO
C2	2-wide	Em ordem	Fora de ordem	—	—	Execução especulativa simples
C3	2-wide	Fora de ordem	Em ordem	16 ent.	—	OoO com ROB de 16 entradas
C4	4-wide	Fora de ordem	Em ordem	32 ent.	—	4-wide OoO; ROB de 32 entradas
C5	4-wide	Fora de ordem	Em ordem	32 ent.	2-bits	Configuração mais realista

Tabela 33: Legenda de Cores Utilizada nos Resultados

Cor	Significado
Exemplo	Melhor valor na linha (max IPC / min stalls)
Exemplo	Pior valor na linha
Exemplo	Rótulo de sequência
Exemplo	Linha par (alternada)
Exemplo	Totais / médias

8 Conclusão

Este relatório experimental permitiu quantificar e validar o impacto dos conflitos de dados e de controle no desempenho de processadores com pipelines de 5 estágios e modelos superescalares avançados. Os ensaios empíricos comprovaram que a cadeia de dependências RAW em ordem degrada severamente a taxa de vazão do sistema, exigindo paradas compulsórias que reduzem o IPC próximo à unidade. Embora mecanismos de adiantamento (*forwarding*) sanem conflitos aritméticos puros, o paradoxo temporal do *load-use* e a irregularidade em desvios condicionais curtos evidenciaram as limitações físicas de abordagens puramente escalares.

As otimizações integradas, contudo, demonstraram a viabilidade de contornar tais gargalos. A reordenação estática por software ocultou com sucesso a latência do *load delay slot*, enquanto a inércia dos contadores saturados de 2 bits mitigou expressivamente a penalidade por descartes de instruções (*flushes*) em loops. Por fim, a transição para arquiteturas superescalares dinâmicas assistidas pelo Algoritmo de Tomasulo provou que a eliminação de dependências falsas (WAR/WAW) via renomeação de registradores e a flexibilização do despacho fora de ordem aproximam o processador de seu paralelismo máximo ideal (ILP). Conclui-se que o equilíbrio sinérgico entre técnicas de compilador e estruturas agressivas de hardware é o pilar indispensável para sustentar o desempenho computacional moderno.

Referências

FLICH, Jose; BERTOZZI, Davide (Ed.). **Designing network on-chip architectures in the nanoscale era**. Boca Raton: CRC Press, 2012.

HAGER, Georg; WELLEIN, Gerhard. **Introduction to high performance computing for scientists and engineers**. Boca Raton: CRC Press, 2010.

HARRIS, David Money; HARRIS, Sarah L. **Digital design and computer architecture**. 2. ed. Waltham: Morgan Kaufmann, 2012.

HENNESSY, John L.; PATTERSON, David A. **Arquitetura de computadores: uma abordagem quantitativa**. 5. ed. Rio de Janeiro: Elsevier, 2014.

PACHECO, Peter S. **An introduction to parallel programming**. Waltham: Morgan Kaufmann, 2011.

PATTERSON, David A.; HENNESSY, John L. **Organização e projeto de computadores: a interface hardware/software**. 4. ed. Rio de Janeiro: Elsevier, 2014.

STALLINGS, William. **Arquitetura e organização de computadores**. 10. ed. São Paulo: Pearson Education do Brasil, 2017.

TANENBAUM, Andrew S.; AUSTIN, Todd. **Organização estruturada de computadores**. 6. ed. São Paulo: Pearson Prentice Hall, 2013.

A Sequências S1–S5 e Dados Brutos

B Transcrição Completa das Análises — S1 a S5

B.1 Sequência S1

- **Caracterização:** A sequência S1 possui uma cadeia contínua de dependências RAW entre instruções consecutivas. O objetivo pedagógico é demonstrar o impacto dos hazards RAW e como o *forwarding* elimina *stalls*.
- **Hazards identificados:**
 - addiu → add: RAW em t0
 - add → add: RAW em t1
 - add → add: RAW em t2
 - add → add: RAW em t3
 - add → add: RAW em t4
 - add → sub: RAW em t5
- **Configuração sem forwarding:** Cada dependência necessita aproximadamente 2 *stalls*. Total aproximado: 12 *stalls*. CPI estimado: 25 ciclos/9 instruções = 2,78.
- **Configuração com forwarding:** Todos os RAW aritméticos são eliminados. Nenhum *stall* permanece. CPI aproximado: 1,44.
- **Predição:** Não há *branches*, portanto os preditores não impactam o desempenho.
- **Speedup:** $2,78/1,44 \approx 1,93$.
- **Conclusão:** O *forwarding* praticamente dobra o desempenho do pipeline.

Tabela 34: Tabela de Hazards RAW – Sequência S1

Instrução Consumidora	Registrador Dependente	Produzido por
add \$t1, \$t0, \$t0	\$t0	addiu
add \$t2, \$t1, \$t0	\$t1	add \$t1
add \$t3, \$t2, \$t1	\$t2	add \$t2
add \$t4, \$t3, \$t2	\$t3	add \$t3
add \$t5, \$t4, \$t3	\$t4	add \$t4
sub \$t6, \$t5, \$t4	\$t5	add \$t5

Tabela 35: Tabela Consolidada – Sequência S1

Configuração	Forwarding	Predição	Stalls	CPI Aproximado
(a)	Não	Não	12	2,78
(b)	Sim	Não	0	1,44
(c)	Sim	Estática	0	1,44
(d)	Sim	1 bit	0	1,44
(e)	Sim	2 bits	0	1,44

B.2 Sequência S2

- **Caracterização:** S2 implementa um loop de soma de vetor. Ela demonstra *load-use hazard*, *branch hazard* e impacto dos preditores.
- **Hazards identificados:**
 1. *Load-use*: $lw \rightarrow add$
 2. Dependência RAW: $addiu \rightarrow bne$
 3. *Hazard* de controle: *branch* do loop.
- **Sem forwarding:** 2 stalls no *load-use*, 2 stalls RAW no *branch*, além de *flushes* de *branch*. CPI aproximado: 2,2.
- **Com forwarding:** RAW aritméticos eliminados, porém o *load-use* permanece com 1 stall inevitável. CPI aproximado: 1,6.
- **Predição estática:** Loop tomado frequentemente. *Always not-taken* gera muitos erros. CPI aproximado: 1,5.
- **Predição dinâmica de 1 bit:** Preditores aprende o comportamento do loop. Taxa aproximada de acerto: 75%. CPI aproximado: 1,3.
- **Predição dinâmica de 2 bits:** Preditores estabiliza em *taken*. Melhor desempenho. CPI aproximado: 1,2.
- **Escalonamento estático:** Reordenação possível ($lw \rightarrow addiu \rightarrow addiu \rightarrow add \rightarrow bne$), o que oculta com sucesso o stall do *load-use*.
- **Speedup:** $2,2/1,2 \approx 1,83$.
- **Conclusão:** *Forwarding* + preditor de 2 bits + escalonamento estático aproximam o pipeline do cenário ideal.

Tabela 36: Tabela de Penalidades – Sequência S2

Tipo	Penalidade
Load-use	1 ciclo
Branch tomado	~2 ciclos

Tabela 37: Pipeline Simplificado – Com Forwarding (S2)

Ciclo	lw	add
1	IF	
2	ID	IF
3	EX	ID
4	MEM	stall
5	WB	EX

Tabela 38: Tabela Consolidada – Sequência S2

Configuração	Stalls	CPI Aproximado
(a) Sem forwarding	Alto	2,2
(b) Forwarding	Médio	1,6
(c) Pred. estática	Médio	1,5
(d) Pred. 1 bit	Baixo	1,3
(e) Pred. 2 bits	Muito baixo	1,2

Tabela 39: Pipeline Simplificado – Com Forwarding (S2)

Ciclo	lw	add
1	IF	
2	ID	IF
3	EX	ID
4	MEM	stall
5	WB	EX

Tabela 40: Tabela do Preditor de 2 Bits – Sequência S2

Iteração	Real	Estado Antes	Predição	Acerto?	Estado Depois
1	T	WNT	NT	Não	WT
2	T	WT	T	Sim	ST
3	T	ST	T	Sim	ST
4	T	ST	T	Sim	ST
5	T	ST	T	Sim	ST
6	T	ST	T	Sim	ST
7	T	ST	T	Sim	ST
8	NT	ST	T	Não	WT

B.3 Sequência S3

- **Caracterização:** S3 foi criada especificamente para demonstrar o *load-use hazard* clássico.
- **Hazards:** $lw \rightarrow add$, $lw \rightarrow sub$, $lw \rightarrow add$. Todos são RAW do tipo *load-use*.
- **Sem forwarding:** Cada dependência necessita de aproximadamente 2 *stalls*. Total aproximado: 6 *stalls*. CPI aproximado: 2,0.
- **Com forwarding:** O mecanismo reduz parcialmente o problema, mas cada *load-use* ainda necessita de 1 *stall* inevitável. CPI aproximado: 1,7.
- **Demonstração temporal:** O lw produz o dado apenas ao final do estágio MEM. A instrução seguinte precisa do dado no início de EX. Logo, o valor chega tarde demais para o consumo imediato.
- **Conclusão:** O *forwarding* não consegue eliminar completamente o *load-use hazard*.
- **Escalonamento:** Inserir instruções independentes entre o lw e a instrução consumidora pode ocultar a latência.
- **Speedup:** $2,0/1,7 \approx 1,18$.

Tabela 41: Tabela Consolidada – Sequência S3

Configuração	Forwarding	Predição	Stalls	CPI Aproximado
(a)	Não	Não	6	2,0
(b)	Sim	Não	3	1,7
(c)	Sim	Estática	3	1,7
(d)	Sim	1 bit	3	1,7
(e)	Sim	2 bits	3	1,7

Tabela 42: Pipeline – Sem Forwarding (S3)

Ciclo	lw	add
1	IF	
2	ID	IF
3	EX	ID
4	MEM	stall
5	WB	stall
6		EX

Tabela 43: Pipeline – Com Forwarding (S3)

Ciclo	lw	add
1	IF	
2	ID	IF
3	EX	ID
4	MEM	stall
5	WB	EX

B.4 Sequência S4

- **Caracterização:** S4 demonstra RAW, WAR e WAW. Ela serve de ponte para o estudo de execução fora de ordem (*Out-of-Order*) e Algoritmo de Tomasulo.
- **Dependências RAW:** $I1 \rightarrow I2$, $I2 \rightarrow I4$, $I3 \rightarrow I5$, $I4 \rightarrow I5$, $I5 \rightarrow I6$.
- **Dependências WAR:** $I2 \rightarrow I5$.
- **Dependências WAW:** $I1 \rightarrow I4$, $I2 \rightarrow I6$.
- **Sem forwarding:** O RAW gera múltiplos *stalls*. A dependência após a multiplicação aumenta consideravelmente a latência. CPI aproximado: 2,0.
- **Com forwarding:** Quase todos os RAW são eliminados. Permanece um possível *stall* de latência após a instrução MUL. CPI aproximado: 1,45.
- **Observação:** Conflitos WAR e WAW normalmente não causam *stalls* em pipelines escalares *in-order*.
- **Renomeação:** A técnica de renomeação elimina as dependências falsas (WAR/WAW), restando apenas as RAW (verdadeiras).
- **Escalonamento:** A reordenação de instruções independentes reduz os *stalls* residuais.
- **Speedup:** $2,0/1,45 \approx 1,38$.
- **Conclusão:** S4 evidencia a importância crucial da renomeação de registradores em arquiteturas modernas.

Tabela 44: Tabela de Hazards – Sequência S4

Tipo	Registrador	Instruções
RAW	\$t1	$I1 \rightarrow I2$
RAW	\$t4	$I2 \rightarrow I4$
RAW	\$t2	$I3 \rightarrow I5$
RAW	\$t1	$I4 \rightarrow I5$
RAW	\$t5	$I5 \rightarrow I6$
WAR	\$t5	$I2 \rightarrow I5$
WAW	\$t1	$I1 \rightarrow I4$
WAW	\$t4	$I2 \rightarrow I6$

Tabela 45: Tabela Consolidada – Sequência S4

Configuração	Forwarding	Predição	Stalls	CPI
(a)	Não	Não	~ 7	2,0
(b)	Sim	Não	~ 1	1,45
(c)	Sim	Estática	~ 1	1,45
(d)	Sim	1 bit	~ 1	1,45
(e)	Sim	2 bits	~ 1	1,45

B.5 Sequência S5

- **Caracterização:** S5 possui múltiplos desvios condicionais. O objetivo central é demonstrar os *hazards* de controle e avaliar os limites dos preditores.
- **Hazards:** *Branch hazards* e *load-use* entre *lw* e as instruções *bltz/beqz*.
- **Padrão do vetor:** Elementos alternando entre negativos, positivos e zeros, gerando um comportamento altamente irregular para algoritmos de predição.
- **Sem forwarding e sem predição:** Ocorrência de múltiplos *flushes*. CPI aproximado: 2,8.
- **Com forwarding:** Os *branches* mal preditos continuam dominando e limitando a eficiência. CPI aproximado: 2,3.
- **Predição estática:** A estratégia *Always not-taken* produz erros severos devido ao perfil dos dados. CPI aproximado: 2,0.
- **Predição dinâmica de 1 bit:** Tenta aprender com o comportamento recente, porém oscila excessivamente em padrões alternados. CPI aproximado: 1,7.

- **Predição dinâmica de 2 bits:** Mais estável, pois reduz mudanças abruptas de estado. CPI aproximado: 1,5.
- **Escalonamento:** A inserção de instruções independentes reduz parcialmente o impacto do *load-use* antes do desvio.
- **Speedup:** $2,8/1,5 \approx 1,87$.
- **Conclusão:** S5 representa o caso clássico de um pipeline severamente limitado por *hazards* de controle.

Tabela 46: Classificação do Vetor de Teste – Sequência S5

Valor	Resultado
-3	negativo
5	positivo
0	zero
2	positivo
-1	negativo
0	zero
7	positivo
-4	negativo
0	zero
3	positivo

Tabela 47: Tabela Consolidada – Sequência S5

Configuração	Hazard Dominante	CPI ...
(a) Sem forwarding	Controle + RAW	2,8
(b) Forwarding	Controle	2,3
(c) Predição estática	Controle	2,0
(d) Predição 1 bit	Controle	1,7
(e) Predição 2 bits	Controle	1,5

Tabela 48: Tabela do Preditor de 2 Bits – Sequência S5

Iteração	Resultado	Estado Antes	Predição	Acerto?	Estado Depois
1	T	WNT	NT	Não	WT
2	NT	WT	T	Não	WNT
3	NT	WNT	NT	Sim	SNT
4	NT	SNT	NT	Sim	SNT
5	T	SNT	NT	Não	WNT
6	NT	WNT	NT	Sim	SNT
7	NT	SNT	NT	Sim	SNT
8	T	SNT	NT	Não	WNT

B.6 Conclusão Geral das Análises

As sequências experimentais S1–S5 demonstraram de forma empírica os principais gargalos e soluções da engenharia de processadores, mapeando de forma prática conceitos fundamentais como os *hazards* RAW, *load-use*, *branch hazards*, WAR e WAW.

A compilação dos dados coletados consolida os seguintes achados fundamentais:

- O mecanismo de **forwarding** reduz drasticamente os *stalls* do tipo RAW aritmético, limpando a execução de dados dependentes.
- Os **load-use hazards** são limitações temporais físicas intransponíveis para o *forwarding* isolado, exigindo paradas parciais pelo hardware.
- Os **preditores dinâmicos** melhoram significativamente a taxa de vazão em estruturas de loops e desvios ramificados.
- Os **preditores de 2 bits** superam as variações de 1 bit pela inércia gerada pelo histórico de contadores saturados, mitigando flutuações anômalas.
- A aplicação de **renomeação de registradores** elimina de forma limpa dependências falsas (WAR e WAW).
- O **escalonamento estático** via software maximiza o paralelismo efetivo a nível de instrução (ILP).

Fica evidente, portanto, que a coordenação sinérgica entre os blocos de *forwarding*, predição de desvios, escalonamento e renomeação arquitetural é o pilar que sustenta o desempenho e a vazão das CPUs modernas de alto desempenho.